

Movientum: A Full-Stack Personalized Movie and TV Discovery Platform with a Graph-Augmented, Gated Learning-to-Rank Recommendation Engine

System Architecture, Recommendation Algorithms, and Design Report

Movientum Engineering Documentation

System Architecture, Machine Learning, and Data Pipeline Design Report

Revised September 2026 to reflect the shipped system

Abstract—Movientum is a movie and television discovery platform built as a React single-page application over a FastAPI modular-monolith backend, with a graph-augmented recommendation engine as its core. Users express preference through a four-category rating vocabulary (*Skip*, *Timepass*, *Go For It*, *Perfection*) and through thumbs, clicks, watch and watchlist actions; every such action updates a per-user taste profile immediately, off the request path. Four recommendation surfaces share that profile: *similar items*, driven by Personalized PageRank over a bipartite title-feature graph, a 16-feature candidate matrix, a composite ranking score, and an XGBoost `XGBRanker` that is admitted to production only when it beats the composite on held-out users; a *For You* feed that merges four seed buckets in fixed proportions and reorders them for quality, taste, novelty and diversity; a *Content DNA* engine that ranks titles against a basket of seeds using IDF-weighted facet similarity; and an LLM surface backed by Gemini with a Groq fallback. Around this core sit a local-first fuzzy search ranker, a Redis-only news pipeline built as atomically swapped snapshot generations with taste-based personalization, tier lists resolved lazily from declarative templates, and a page-bundle caching layer that serves each rendered page from a single Redis key. The entire system runs on free-tier infrastructure (Vercel, Azure App Service with a Render failover, Supabase PostgreSQL, Upstash Redis), so every performance property is obtained in code rather than through hardware. This report describes the architecture, the algorithms and their parameters, and the design trade-offs behind them.

Index Terms—recommendation systems, learning-to-rank, XGBoost, personalized PageRank, random walk with restart, team-draft interleaving, maximal marginal relevance, IDF weighting, FastAPI, PostgreSQL, Redis, cache stampede, modular monolith, news personalization

I. INTRODUCTION

A. Motivation

Numeric rating scales are ambiguous: a “7/10” can mean enthusiasm from one user and indifference from another. Movientum replaces them with four emotionally unambiguous words and pairs that vocabulary with a recommendation pipeline that reacts to every interaction immediately rather than after a batch job.

B. Design Goals

G1. **Unambiguous preference capture** through a categorical rating vocabulary and explicit thumbs.

- G2. **Immediate personalization**: every signal moves the user’s taste profile before their next request, without blocking the response that carried it.
- G3. **Explainable ranking**: graph proximity plus interpretable features and gradient-boosted trees rather than an opaque embedding model, so every recommendation can be traced to shared edges and feature contributions.
- G4. **Zero infrastructure spend**: the platform runs on free tiers. Speed must come from code—caching, compression, deferral of non-critical work, and moving CPU-bound work off the event loop—never from added workers or larger instances.
- G5. **Safe model deployment**: a learned model may only influence ranking after it has been shown to outperform the non-learned baseline.

C. Report Structure

Section II presents the system architecture; Sections III–IV the backend and frontend designs; Section V the data model. Sections VI–XI cover the recommendation engine: graph, ranking, the For You feed, the feedback loop, Content DNA, LLM recommendations, and nightly retraining. Sections XII–XIV cover search, news, and tier lists. Sections XV–XVIII cover caching, deployment, performance, and observability. Section XIX discusses trade-offs.

II. SYSTEM ARCHITECTURE

A. Architectural Philosophy

Movientum is a **modular monolith**: one backend code-base with strict internal layering rather than a microservice mesh. The deciding factor is the recommendation graph, a `networkx` object held in the API process’s memory. Splitting the engine into its own service would put a network hop—or a graph database—inside every random walk, and each walk touches many neighbours.

B. Tiers

- 1) **Client**: React 19 single-page application on Vercel’s CDN.
- 2) **API**: FastAPI container on Azure App Service, with an identical container on Render as a failover target.

- 3) **Background:** Celery worker and beat scheduler built from the same image; the same job coroutines can also run inside the API process when an administrator triggers them.
- 4) **Data:** Supabase PostgreSQL for durable state; Upstash Redis for response caching, page bundles, the JWT blacklist, the news snapshot, and the Celery broker.

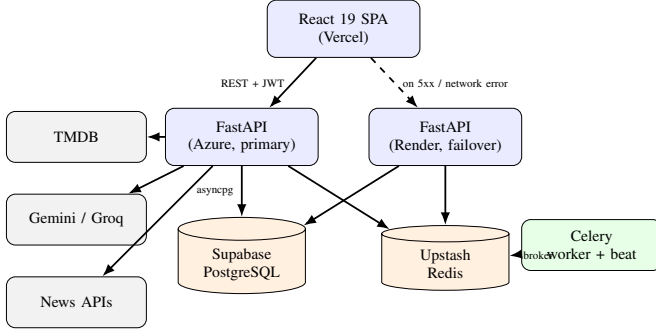


Fig. 1: Deployed topology. The frontend fails over to the Render backend when the primary returns a network or 5xx error.

C. Request Path

Every request follows the same shape: the React client calls the API through a single HTTP client; a router authenticates and validates it; a service consults Redis; on a miss it reads PostgreSQL and calls TMDB only for titles absent locally; the result is cached. Titles fetched from TMDB for the first time are persisted and, after the response has been sent, ingested into the recommendation catalogue and spliced into the live graph.

D. External Services

TMDB supplies all title, person, credit, video and streaming-provider metadata. NewsAPI, Currents and ApiTube supply news. Gemini (with Groq fallback) powers LLM recommendations. AniList and Fandom supply animated characters for tier lists. Google Identity Services provides sign-in. Azure Application Insights receives OpenTelemetry traces and metrics.

III. BACKEND ARCHITECTURE

A. Layering

The backend enforces three layers:

- **Routers** handle HTTP only: parameters, authentication dependencies, Pydantic validation.
- **Services** hold business logic: caching, TMDB calls, scoring, recommendation.
- **Repositories and ORM models** hold SQL.

Twenty-six routers are registered explicitly under `/api/v1/`. Within a router, static paths (e.g. `/trending`) are declared before parameterised ones (`/movie_id`), because FastAPI matches in declaration order.

B. Asynchronous Design

The stack is asynchronous end to end (`asyncpg`, `async Redis`). Three techniques keep the event loop responsive:

- 1) **Deferral:** taste-profile updates, catalogue ingestion, signal processing and suppression run in FastAPI `BackgroundTasks` after the response is sent.
- 2) **Off-loop CPU work:** graph walks, matrix construction, ranking and image re-encoding run in `asyncio.to_thread`.
- 3) **Round-trip consolidation:** independent reads that share a session (e.g. watched, watchlisted, low-rated and suppressed titles) are merged into one `UNION` query.

C. Startup

Uvicorn does not open its port until the application lifespan yields, and an Azure cold start keeps the first visitor waiting for exactly that interval. The lifespan therefore performs only the PostgreSQL and Redis connectivity checks inline. Everything else is spawned as background tasks after the port opens: raw-SQL creation of four operational tables, a single `content_catalog` read that builds both the recommendation graph and the Content DNA index, deletion of unreferenced low-popularity movies older than 30 days (and expired suppressions), and one TLS handshake to TMDB so the first user does not pay for it.

D. Authentication

Authentication is stateless JWT. Access tokens live 48 hours and refresh tokens 7 days; each carries a unique `jti`. Logout writes the access token's `jti` to Redis with a TTL equal to its remaining lifetime; refresh rotation blacklists the old refresh `jti`. Every decode checks signature, expiry and blacklist. Three FastAPI dependencies express access levels: required, optional (personalise when present), and admin; the admin dependency re-reads the role from the database so role changes apply without a new login. Besides email and bcrypt-hashed passwords, users can sign in with Google (ID token verified against Google's JWKS, cached 6 h, then login, link-by-email, or create) or with a previously registered device identifier.

E. Background Jobs

Celery beat (IST) runs TMDB sync at 03:00, ranker re-training at 03:30, the news title-index build at 03:45, the new-episode check at 04:00, and per-region trailer refreshes every three hours, staggered by ten minutes. Because beat is not reliably resident on free hosting, the administrator panel can invoke every job through `/internal/trigger/{task}`, which runs the same coroutine inside the API process with progress and cancellation state. News ingestion has no schedule at all; it is administrator-triggered only.

IV. FRONTEND ARCHITECTURE

A. Rendering Strategy

The client is fully client-side rendered. Nearly every view is personalised, so server rendering would offer little cacheable output, while client state lets the interface respond to a thumbs press before the server does.

B. Single HTTP Client

All traffic passes through one Axios instance that (i) injects the bearer token, (ii) on a 401 pauses concurrent requests, refreshes once, and replays the queue, (iii) allows 120 s per request to absorb 15–30 s cold starts, (iv) retries a failed request against the secondary backend, and (v) raises a non-blocking “still browsable” notice when both backends fail. Tokens are stored in `localStorage` or `sessionStorage` according to the “Remember me” choice.

C. Perceived Performance

- **Page bundles:** home, movie, TV, person and dashboard pages each load from one `/pages/*` request (Section XV).
- **Home snapshot:** the last home bundle is kept locally for 24 h and painted immediately while the backend wakes.
- **Deferred sections:** below-the-fold rows mount, and therefore fetch, only when they approach the viewport.
- **Hide-and-replace:** a thumb-down card is removed and a buffered replacement slides in, so feedback never leaves a hole.
- **Code splitting:** React, Motion and OGL ship as separately cached chunks.

V. DATA MODEL

A. Unified Title Key

Movies and television shows share the `movies` table under the composite key `(id, type)`, because TMDB reuses numeric identifiers across the two media types. Every title-referencing table uses the same composite key.

B. Storage Discipline

News is never written to PostgreSQL. Recommendation features are stored as integer arrays and JSONB rather than join tables. Four operational tables are created by raw SQL at startup, outside Alembic. Unreferenced, low-popularity catalogue rows are pruned after 30 days.

VI. RECOMMENDATION ENGINE: GRAPH AND RANKING

A. Surfaces

TABLE II: Recommendation Surfaces

Surface	Engine
Similar items	PPR + features + ranker + baseline blend
For You	Similar-items engine per seed bucket, merged
Content DNA	IDF facet similarity over a basket
AI picks	Gemini with Groq fallback

All four read, and all four write back into, the same `user_taste_profiles` row.

B. Bipartite Content Graph

The graph has one node per catalogue title and one node per feature value (a genre, keyword, person, studio, era or language). A title connects to each of its features with a weight reflecting how discriminative that feature type is (Table III).

TABLE III: Graph Edge Weights

Edge	Weight	Rationale
Director	2.5	Strongest creative signal
Studio	1.8	Anime only; house style
Keyword	1.5 (2.4 anime)	Theme beyond genre
Genre	1.0	Baseline similarity
Cast	0.8	Talent signal
Era	0.6	Decade taste
Language	0.4	Weak but useful

Anime receives special treatment. Walks from an anime seed previously escaped through the two largest hubs in that corpus—the Animation genre and the Japanese language—and landed on arbitrary animated titles. The anime reweighting lowers the Animation-genre edge to 0.35, raises keyword edges to 2.4, and adds studio edges, omitting any studio shared by more than 250 anime because such a node fans a walk across an entire output catalogue. The graph is a process-global `networkx` [8] object, built at startup, and extended in place when a title is ingested on demand; a per-graph-version cache of the PageRank transition matrix avoids rebuilding it per request.

C. Personalized PageRank

Given graph $G = (V, E)$ and seed s , the Personalized PageRank [2], [3] vector π is the stationary solution of

$$\pi = dW\pi + (1 - d)e_s, \quad (1)$$

where W is the column-stochastic transition matrix derived from the weighted adjacency of G , $d = 0.85$ is the damping factor (restart probability 0.15), and e_s is the indicator of the seed. π_v is the long-run probability that a walker who repeatedly teleports back to s is at v . Title nodes with the highest π_v (about 100) become candidates. Unlike flat cosine similarity, PPR captures multi-hop relationships: two titles sharing neither genre nor cast still score highly if both connect strongly to the same director and several of the same keywords.

D. Candidate Guards

Before scoring, candidates are filtered for the seed itself, adult content, and poor-quality or strongly mismatched items.

E. The 16-Feature Matrix

Each candidate is described by sixteen features in four families (Table IV).

TABLE I: Principal Tables by Role

Table(s)	Role
movies, genres, directors	Browsable title catalogue with a TSVECTOR and a trigram-indexed normalised title
content_catalog	Recommendation features per title: genre, keyword, cast (top 10), crew by role, studio, language, origin countries, era, votes, popularity
user_taste_profiles	Per-user JSONB weight maps for genre, cast, crew, keyword, language, era, plus negative weights
interaction_log	Every feedback event with its graded label, surface, and exact 16-feature snapshot; ranker training data
rec_suppression	Titles dismissed by the user, with a decaying weight and a 90-day expiry
ratings, movie_ratings, tv_ratings	User category ratings and imported seed distributions for the rating meter
watch_history, watchlist, watchlist_collections, watchlist_items	Library; history and the single watchlist are taste signals, collections are the primary list UX
ai_rec_memory, ai_rec_sessions	Upsert-by-title thumbs on LLM suggestions (fed back into prompts) and a request audit
tier_lists	Saved tier boards as JSONB with frozen item metadata and a public share id
people	Local cast/crew search index
watching_tracker, notifications	Followed shows and user alerts

TABLE IV: Candidate Feature Matrix

Family	Features	Meaning
Graph	ppr_score, ppr_rank_norm	Walk probability and its normalised rank
Content	vote_average, vote_count_log, popularity_log, recency_score	The title's own quality, confidence, popularity, recency
Personal	user_genre_score, user_cast_score, user_crew_score, user_keyword_score, user_era_score, user_language_mult	The user's taste weights intersected with the title's features
Overlap	genre_overlap_count, cast_overlap_count, same_language, same_era	Direct similarity to the seed

F. Composite Score and Gated Model Blend

Ranking always starts from a composite score:

$$c = 0.45g + 0.30q + 0.25t, \quad (2)$$

where g is ppr_rank_norm; $q = \frac{\bar{r}}{10} \cdot \min(1, \frac{\log(1+n)}{\log(1+10^4)})$ combines rating $\bar{r}$ with vote-count confidence n ; and $t \in [0, 1]$ is the mean of the six personal features, each normalised by the user's own peak, rescaled so that 0.5 is neutral (guests receive exactly 0.5).

An XGBRanker [1] is consulted only when its metadata file marks it *approved* (Section XI). It is then blended at the level of ranks, so neither score's scale matters:

$$s_i = 0.5 \text{rank}(c)_i + 0.5 \text{rank}(m)_i, \quad (3)$$

with ties resolved by walk order. The model is never used alone, because training snapshots do not record true graph position and so the model cannot judge similarity by itself.

This gate replaced an earlier fallback to the raw ppr_score, and it exists because of a concrete failure:

a model trained on about 100 interactions from two users had early-stopped at iteration zero, collapsing to one shallow tree keyed on two taste features. For a guest every candidate received the same score, and a descending sort returned the *least* similar titles of the walk.

G. Baseline Blend: Team-Draft Interleaving

The ranked list A is interleaved with a simpler genre-affinity baseline B using Team-Draft Interleaving [5]: slots are allocated deterministically ($\lfloor 0.7k \rfloor$ to A , the rest to B for $k = 100$), the draw order is shuffled to remove position bias, and duplicates (by (id, type)) are skipped with graceful fill from the other list. Interleaving decides *membership* only.

H. Final Ordering

Membership fixed, the list is reordered by

$$f = 0.40\rho + 0.30Q + 0.20T + 0.10R, \quad (4)$$

where ρ is the relevance carried from the ranker (discounted by 0.85 for baseline-sourced items so the 70/30 intent sur-

vives), R is recency, T the taste match, and Q a Bayesian quality score

$$Q = \frac{v}{v+m} \cdot \frac{\bar{r}}{10} + \frac{m}{v+m} \cdot \frac{p}{10}, \quad (5)$$

with v votes, prior $p = 6.5$, and $m = 300$ ($m = 90$ for anime, whose vote counts are an order of magnitude lower). Without shrinkage, a 9.0 with 40 votes would outrank an 8.2 with 9,000. Anime seeds use a dedicated formula that adds an anime-affinity term (franchise, keyword, studio and crew overlap) and discounts non-anime candidates. Finally the user's watched, watchlisted, low-rated and suppressed titles are excluded in the single consolidated query noted in Section III.

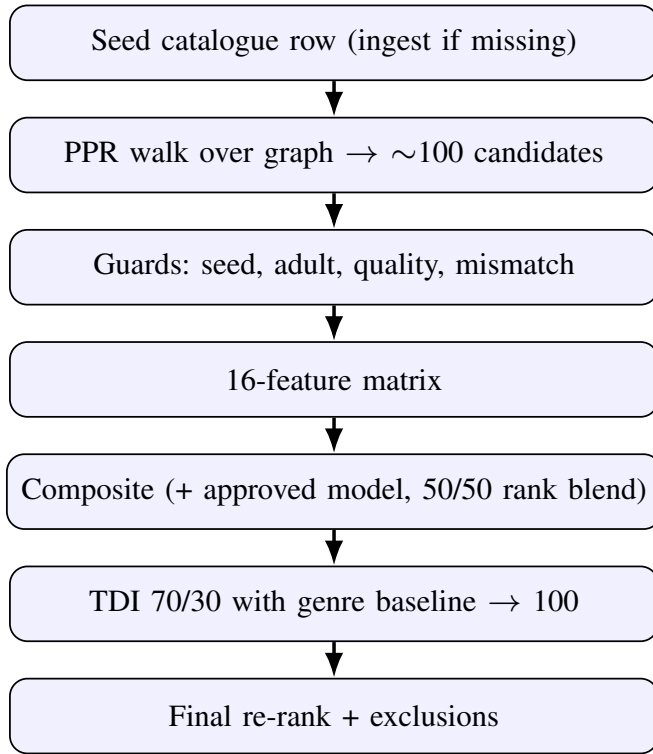


Fig. 2: Similar-items pipeline.

Similar-item results are cached for 6 h per user and 24 h for guests.

VII. THE FOR YOU FEED

A. Diversified Seeds

A feed seeded from a single recent title lets one binge dominate. The feed therefore draws seeds from watch history and the watchlist in buckets, runs the similar-items engine per bucket, and merges the pools in fixed proportions (Table V). Seed choice rotates every 15-minute window so the feed changes between visits.

TABLE V: For You Seed Buckets

Bucket	Share	Purpose
Established	65%	Seeds matching long-term genre weights
Diverse	17.5%	A weakly matching engaged title; re-vives older interests
Recent	12.5%	The latest watch or save
Fresh	7.5%	Popular titles; pure exploration

An N -way weighted interleave allocates slots proportionally and deduplicates. Any shortfall is padded from the genre baseline.

B. Ordering and Diversity

The pool is reordered by

$$f = (0.28 \rho + 0.26 Q + 0.36 T + 0.10 N) \cdot \epsilon, \quad (6)$$

where N is novelty and ϵ is an era multiplier derived from the eras the user has recently engaged with. A greedy maximal-marginal-relevance pass [6] then spreads clusters: scanning a 25-item window, each candidate's score is reduced by 0.12 per earlier pick sharing its genre/language signature (capped at three), and the best adjusted item is taken. The pass only reorders, so the bucket proportions are preserved exactly. Finally, once the share of the user's dominant language exceeds a per-user threshold (default 0.70), at least three other-language titles are guaranteed.

C. Caching and Fallbacks

Pools are cached per 5-page block for 15 minutes, dropping to 3 minutes immediately after a library change so the feed visibly reacts. Users without history receive the baseline feed; guests receive regional TMDB trending.

VIII. THE FEEDBACK LOOP

A. Signals

Every signal applies per-dimension deltas to the taste profile and is logged with a graded label (Table VI).

Explicit signals never decay. Clicks are weighted by $e^{-\lambda \Delta t}$ with $\lambda = 0.01$ per day (half-life ≈ 69 days). Thumbs down and un-watched also accumulate into a negative-weight map used as a soft penalty in taste matching. Weights are not clamped; scorers normalise by the user's own observed peak, so a heavily watched genre keeps rising. Category ratings are deliberately a separate channel: they update the rating meter and exclude low-rated titles from feeds but do not move taste weights.

B. Label Scale

XGBRanker's `rank:ndcg` objective requires non-negative graded relevance. An earlier scale placed thumbs down at -1 and floored it to 0, which made an explicit dislike train identically to removing a title from a watchlist. The scale was shifted so that dislike (1) sits strictly below neutral (2). Because dislikes are rare relative to clicks, their sample weight is multiplied by 1.5 during training.

TABLE VI: Signal Deltas and Training Labels

Signal	Genre	Cast	Crew	Era	Keyword	Language	Label
Watched	+15	+10	+10	+10	+8	+2	5
Thumbs up	+10	+10	+10	+10	+5	+1	4
Added to watchlist	+8	+5	+5	+5	+4	+1	4
Click	+2	—	—	—	+1	+0.2	3
Removed from watchlist	−8	−5	−5	−5	−4	−1	2
Un-watched	−15	−10	−10	−10	−8	−2	2
Thumbs down	−15	−15	−15	−15	−8	−1.5	1

C. Suppression

A thumbs down on a card also inserts the title into `rec_suppression`. Its weight decays as $e^{-0.0256 \Delta t}$, from 1.0 to about 0.1 at the 90-day expiry, rather than dropping off a cliff. Suppression is enforced as a fourth arm of the exclusion UNION, so it costs no extra round trip, and expiry requires no scheduled job because reads filter on `expires_at`.

D. Request Path

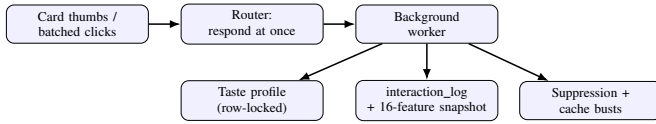


Fig. 3: Feedback processing. Clicks are batched client-side (up to 20 per request); all writes happen after the response.

Each event records its originating surface. LLM-card thumbs additionally upsert `ai_rec_memory`, so the prompt never re-suggests that title. News thumbs are scoped to news and never touch the taste profile.

IX. CONTENT DNA: BASKET RECOMMENDATIONS

A. Goal

Given a basket of titles, find titles whose combined “DNA”—keywords, genres, crew, cast and studio—is closest. The engine is split into pure modules (IDF table, profile, retrieval, scoring) that have no database, cache or network dependency, and one orchestration module that owns I/O. The pure modules are therefore replayable offline against a catalogue snapshot.

B. IDF Weighting

Rare features carry more information than common ones: sharing the keyword *time loop* says more than sharing *Drama*. Each feature f is weighted by smoothed inverse document frequency [7] over the catalogue of N titles, normalised to $[0, 1]$:

$$\text{idf}(f) = \log \frac{N}{1 + \text{df}(f)}, \quad w(f) = \max\left(\frac{\text{idf}(f)}{\log N}, 0\right). \quad (7)$$

The basket’s titles are merged into a single IDF-weighted profile vector per facet, and the dominant language and media type are recorded.

C. Pipeline

- 1) **Recall**: an inverted index returns every title sharing strong features with the profile, with per-facet caps on the number of driving features and a backfill to at least 200 candidates with at least 20 votes.
- 2) **Graph**: bounded PPR from each basket title, cached per title for 24 h.
- 3) **Scoring**: per-facet cosine similarity between candidate and profile, combined with graph, quality and popularity terms (Table VII).
- 4) **Gates**: soft language and media-type gates push off-pattern titles down without removing them.
- 5) **Diversity**: MMR selection avoids a page of one franchise.

TABLE VII: Content DNA Score Weights

Component	Weight
Keyword cosine	0.30
Genre cosine	0.22
Crew (director/writer)	0.12
Cast	0.10
Studio	0.06
Graph (PPR)	0.12
Quality (Bayesian)	0.05
Popularity (tiebreak)	0.03

D. Interactive Extensions

Server-side filters and sorts run over the cached ranked pool; a *negative basket* subtracts matches to “less like this” DNA; logged-in users receive a taste-affinity blend unless they set `ignore_taste`; and a fingerprint of the basket’s top features plus per-result “why” chips make each match explainable. The full 300-item order is computed once per basket and cached for 30 minutes, then sliced into pages of 20.

E. Evaluation

A golden set of 15 curated baskets, five held out, drives leave-one-out, purity, intra-list diversity, popularity and coverage metrics. A 40-configuration random sweep appeared to improve a keyword-heavy variant by about 90% on auto-derived training groups, but moved the held-out baskets by −24%, −3% and +4% on three unseen seeds. The cause was that 24 of 26 training groups were director filmographies, which the crew facet solves by construction. The hand-chosen weights were retained.

X. LLM RECOMMENDATIONS

The LLM surface builds a prompt from the seed title’s genres, keywords and overview, and, for signed-in users, from their taste profile and prior AI thumbs. It calls Gemini off the event loop, falls back through a list of Groq models if Gemini fails, repairs malformed JSON, and resolves every suggestion to a real TMDb title before returning it. Reruns, a focus genre, and “more like these” constraints alter the prompt. The surface shares the taste profile but no graph or ranker code.

XI. NIGHTLY RANKER RETRAINING

A. Procedure

- 1) Load `interaction_log` rows from the last 30 days that carry a feature snapshot.
- 2) Build X from the 16 snapshot features and y from the signal-type label; group rows by user so ranking is learned within one person’s choices; weight samples by time decay and dislikes by 1.5.
- 3) Skip if fewer than 20 rows or fewer than 5 users.
- 4) Hold out the last 15% of user groups.
- 5) Fit `XGBRanker(objective="rank:ndcg", tree_method="hist")` with early stopping on NDCG@10.
- 6) Approve only if at least 5 users, 200 rows and 2 validation users exist *and* the blended model beats the composite alone on held-out NDCG@10.
- 7) If approved, write `ranker.json` and `ranker_meta.json` and hot-reload in process; otherwise leave the live ranking untouched.

B. Objective

$$\text{NDCG}@k = \frac{\text{DCG}@k}{\text{IDCG}@k}, \quad \text{DCG}@k = \sum_{i=1}^k \frac{2^{\text{rel}_i} - 1}{\log_2(i+1)}, \quad (8)$$

with $\text{rel}_i \in \{0, \dots, 5\}$ from Table VI. Optimising NDCG [4] rewards correct ordering at the top of the list, which is where users look.

XII. SEARCH

Search is local-first and deliberately uncached. The instant, autocomplete and results endpoints share one ranker:

- 1) **Normalise**: lowercase, strip accents, punctuation, surplus whitespace and leading articles.
- 2) **Retrieve**: `pg_trgm` similarity on a stored, GIN-indexed normalised title column in both the recommendation catalogue and the browse catalogue, plus full-text search over overviews; people come from a local people index.
- 3) **Score**: a text-relevance function tolerant of typos, prefixes, spacing and word order (`rapidfuzz` ratios), plus a small popularity prior as tiebreak.
- 4) **Fallback**: TMDb is queried only when the best local relevance is weak. People found there are written into

the local index in the background, so repeat searches stay local.

Candidate changes are evaluated with `hit@1`, `hit@3` and `MRR@5` against labelled content and cast/crew queries.

XIII. NEWS

A. Snapshot Generations

News lives entirely in Redis. An administrator-triggered job fetches NewsAPI (50), Currents (600) and ApiTube (50) articles concurrently, deduplicates them by canonical URL, title fingerprint and Jaccard near-duplicate detection to roughly 500, and enriches each article with:

- **Categories**: a rule-based scorer (word-boundary phrases, headline weight, outlet domain, linked-title evidence; at least three points, at most three tags), refined by per-category TF-IDF and logistic-regression models trained on the batch itself from the rule hits, which add missed tags and veto weak ones.
- **Entity links**: headlines matched against a nightly title index so an article links to exact catalogue titles.
- **Taste facets**: TMDb genre ids from categories plus genre, person and keyword ids of linked titles.
- **Quality score**.

The batch is written as generation $N+1$ under `news:v3:{N+1}:*` (article hash; latest, quality, category and entity sorted sets; metadata). A single pointer key is then swapped and the old generation deleted after a 10-second grace period. Readers resolve the pointer first, so a build is invisible until the swap; page caches live inside the generation, so the swap invalidates them all at once. A build lock prevents concurrent builds, and an empty fetch is refused rather than replacing a good snapshot.

B. For You Scoring

For users with at least five interactions, each of the newest 500 articles is scored against a max-abs-normalised taste vector and the user’s rated, watched and watchlisted titles (Table VIII).

TABLE VIII: News For You Weights

Component	Weight
Genre match	0.30
Person match	0.20
Linked title in user library	0.20
Keyword match	0.10
Source tier	0.12
Recency	0.08
Negative taste	−0.15
Already seen	−0.25

A greedy source/category diversity pass keeps 120 identifiers, cached for 10 minutes per user and generation so pagination is stable. Trending is computed on demand from views per hour. News feedback adjusts only news-scoped source, category and entity weights in Redis.

XIV. TIER LISTS

Tier-list templates are declarative recipes, not stored lists: each of 515 templates across 16 categories points at a slice of TMDB (a franchise collection, a studio’s output, a director’s filmography, a show’s cast, a season’s episodes, or the user’s own library), using ten source kinds. The catalogue page performs no I/O—cover posters and item counts are baked into generated modules—and a template resolves only when opened, then caches for 7 days. Character boards select a source per title: AniList for Japanese animation, a Fandom wiki for other animation, and TMDB cast otherwise or on empty results. Saved boards freeze each item’s title and poster so shared links render without calling TMDB. On the client, dragging is implemented on Pointer Events because HTML5 drag-and-drop does not fire on touch screens, and PNG export draws the board directly on a canvas using CORS-enabled poster requests to avoid tainting it.

XV. CACHING ARCHITECTURE

A. Cache-Aside with Stampede Protection

```

1 cached = await get_cached(key)
2 if cached: return cached
3 async with inflight_lock(key) as waited:
4     if waited:
5         cached = await get_cached(key)
6         if cached: return cached
7     data = await fetch()
8     await set_cached(key, data, TTL)

```

When many requests miss the same key, only the first fetches; the rest wait on an in-process event and read the populated key. All keys are built by central functions and carry a version segment, so a response-shape change bumps the version instead of requiring a purge. Writes bust exactly the keys they affect.

B. Page Bundles

Each rendered page is served from one Redis key. A detail page is split into a *static* half shared by every viewer (details, videos, credits, collection, rating distribution; 7 days, busted when anyone rates) and a *per-user* half (library status, similar items; 6 hours). On a bundle miss, the page is rebuilt from its section caches with a single MGET.

TABLE IX: Representative Lifetimes

Data	TTL
TMDB detail, credits, videos, providers	7 d
Trending	10 h
Home bundle	30 min
Similar items (user / guest)	6 h / 24 h
For You pool	15 min (3 min volatile)
Taste profile	2 min
Dashboard lists	48 h, busted on write
DNA order	30 min
Search	not cached

XVI. DEPLOYMENT AND RESILIENCE

One Docker image serves as API, Celery worker, or Celery beat depending on its command. Dependencies are installed in a layer preceding the code copy, and a per-build argument forces a fresh code layer. The primary backend runs on Azure App Service and the same image runs on Render; the client fails over on network or 5xx errors, including during token refresh. Beat must run as a single instance to avoid duplicate jobs, and because it is not reliably resident on free hosting, administrator triggers are the dependable execution path.

XVII. PERFORMANCE UNDER A ZERO-BUDGET CONSTRAINT

Adding workers, larger instances, keep-alive pings or recurring warm-up jobs is ruled out. The techniques in use are therefore all code-level: gzip for responses over 1 KB; page bundles; deferral of side effects to background tasks; off-loop CPU work; consolidated exclusion queries; a cached PageRank matrix per graph version; seven-day caching of static TMDB data; local persistence of fetched titles to avoid TMDB rate limits; a two-phase startup; and client-side snapshots, deferred mounting and chunk splitting.

Two limits are accepted rather than engineered around: the 15–30 s cold start of an idle Azure container, and eviction under memory pressure on the free Redis tier, which may drop any key regardless of its TTL. Everything stored in Redis is therefore rebuildable. A standing rule governs optimisation work: it may change *when* or *where* a recommendation is computed and how long it is cached, but never *which* items are returned or their order. A before/after harness enforces this by failing when ordered recommendation identifiers change. Changes intended to improve quality are permitted only with a measured improvement and no added request latency.

XVIII. OBSERVABILITY

The backend exports OpenTelemetry traces, metrics and logs to Azure Application Insights, degrading silently when no connection string is configured. Custom instruments cover PPR walk time, feature-matrix build time, ranker inference time and cold-start count; graph node and edge counts and rebuild time; retrain outcome (success, skipped for insufficient data, skipped as not approved, error), duration, rows and best iteration; feedback signal counts and taste-update latency; cache hits and misses by key family; and Celery task counts and durations. Grafana dashboards are generated from this instrument list. A health endpoint reports database and cache status and returns 503 when either is unavailable.

XIX. DISCUSSION: DESIGN TRADE-OFFS

A. Modular Monolith versus Microservices

Independent service scaling is traded for in-process graph locality and one deployable. Given a catalogue of tens of thousands of titles and walks that touch many neighbours, a network hop per traversal would cost more than any scaling benefit. The cost is that each process holds its own graph copy, which is one reason the system runs a single worker.

B. Graph plus Trees versus Deep Recommenders

Personalized PageRank with an interpretable feature matrix and gradient-boosted trees needs no GPU, trains nightly on modest CPU, and makes every recommendation traceable. A two-tower or sequence model would offer more representational capacity at the cost of infrastructure and explainability that this platform cannot afford.

C. Gated versus Always-On Learning

Shipping the learned ranker unconditionally produced measurably worse results when data were scarce. The approval gate, the rank-level blend and the composite fallback make the learned component strictly additive: it can only take effect after demonstrating improvement on held-out users.

D. Membership versus Order

Interleavers and bucket mixers decide *which* items appear; separate scorers decide *where*. Keeping the two concerns apart preserves the intended blend proportions while still placing the best items first, and allows diversity passes that reorder without disturbing the mix.

E. Snapshot News versus Continuous Ingestion

Continuous crawling consumed paid-API quota and required expiry jobs and rollups. A single daily snapshot with atomic generation swap costs about 36 provider requests per run, needs no expiry job, and guarantees readers never observe a partially built feed.

F. Real-Time Profiles with Nightly Models

Taste profiles update on every interaction, so the next request reflects the latest behaviour, while the ranker retrains once per day. This captures most of the responsiveness of online learning without retraining a model in the serving path.

XX. CONCLUSION

Movientum shows that a responsive, explainable personalization system can be built on a modular monolith and free-tier managed infrastructure. Its core—a weighted bipartite graph walked by Personalized PageRank, an interpretable 16-feature matrix, a composite score that a learned ranker may join only after proving itself, and a membership/ordering separation that preserves blend intent—is complemented by an IDF-weighted basket engine, an LLM surface, a snapshot-based news pipeline personalised from the same taste profile, and a caching design built around page bundles and stampede protection. Each decision favours measured quality, explainability, and zero operational spend.

REFERENCES

- [1] T. Chen and C. Guestrin, “XGBoost: A Scalable Tree Boosting System,” in *Proc. 22nd ACM SIGKDD Int. Conf. on Knowledge Discovery and Data Mining*, 2016, pp. 785–794.
- [2] L. Page, S. Brin, R. Motwani, and T. Winograd, “The PageRank Citation Ranking: Bringing Order to the Web,” Stanford InfoLab, Tech. Rep. 1999-66, 1999.
- [3] H. Tong, C. Faloutsos, and J.-Y. Pan, “Fast Random Walk with Restart and Its Applications,” in *Proc. 6th Int. Conf. on Data Mining (ICDM)*, 2006, pp. 613–622.
- [4] K. Järvelin and J. Kekäläinen, “Cumulated Gain-Based Evaluation of IR Techniques,” *ACM Trans. Inf. Syst.*, vol. 20, no. 4, pp. 422–446, 2002.
- [5] F. Radlinski, M. Kurup, and T. Joachims, “How Does Clickthrough Data Reflect Retrieval Quality?,” in *Proc. 17th ACM Conf. on Information and Knowledge Management (CIKM)*, 2008, pp. 43–52.
- [6] J. Carbonell and J. Goldstein, “The Use of MMR, Diversity-Based Reranking for Reordering Documents and Producing Summaries,” in *Proc. 21st ACM SIGIR Conf.*, 1998, pp. 335–336.
- [7] K. Spärck Jones, “A Statistical Interpretation of Term Specificity and Its Application in Retrieval,” *Journal of Documentation*, vol. 28, no. 1, pp. 11–21, 1972.
- [8] A. Hagberg, P. Swart, and D. S. Chult, “Exploring Network Structure, Dynamics, and Function using NetworkX,” in *Proc. 7th Python in Science Conf. (SciPy)*, 2008, pp. 11–15.
- [9] S. Ramírez, “FastAPI: Modern, Fast (High-Performance) Web Framework for Building APIs with Python,” *FastAPI Documentation*, 2023. [Online]. Available: <https://fastapi.tiangolo.com>
- [10] PostgreSQL Global Development Group, “PostgreSQL Documentation: Chapter 12, Full Text Search,” *PostgreSQL 15 Documentation*, 2023.